Homework 1: Image Enhancement

Computer Vision, Fall 2026 · Mattia Bortolaso
Code and results: https://github.com/Borto71/computer-vision-hw1

1. Objective

I built a small pipeline of classical methods (OpenCV and NumPy only, nothing learned) to enhance colour photos with low light, low contrast and noise. I compared the methods with metrics and by eye, mainly to see what each one helps with and what it breaks.

2. Images and degradation

I used five photos (Table 1), one of mine and four from Pexels. They cover fog and low contrast (img1, img3), strong backlight (img2, img5) and lamps next to deep shadows (img4). Each one is resized to 1024 px on the longest side with `INTER_AREA`.

image	content	condition	source	license
img1	Foggy street at night, seen from a window	low light, fog, light sources	Personal photo	Own work
img2	City skyline in silhouette at sunset	backlight, very dark buildings	Pexels, photo 18927434, Oscar Portan	Pexels License
img3	Foggy road with bare trees	low contrast, fog	Pexels, photo 34649347, Christina & Peter	Pexels License
img4	Night market fruit stall under lamps	non-uniform light, dark shadows	Pexels, photo 34559646, Jahra Tasfia Reza	Pexels License
img5	Dark living room, backlit by a window	non-uniform light, dark shadows	Pexels, photo 30422461, Try Putro Utomo	Pexels License

Table 1: images used, with source and license. Full links are in [6].

Each image also gets a controlled degradation. With I the original in float and μ its mean over all pixels and channels:

$$I_{\text{deg}} = \text{clip}([\, \alpha\,(I - \mu) + \mu \,] \cdot r(x) + n,\ 0,\ 255), \quad n \sim \text{N}(0, \sigma^2)$$

Here $\alpha = 0.5$ halves the contrast (one μ for all channels, so colours don't shift), $r(x)$ is a horizontal ramp from 0.7 on the left to 1.0 on the right, and $\sigma = 10$ is independent noise per channel. The seed is `default_rng(42 + i)`, with i the image index in alphabetical order. Files are saved as PNG because JPEG would alter the noise. **The originals are the reference for the metrics on these synthetic degradations. In a real scenario a clean reference is usually not available**, so I also ran every method on the real originals and judged those by eye.



Figure 1: the five images (top) and their degraded versions (bottom). The left side is darker because of the illumination ramp.

3. Methods

All methods take the degraded 8-bit BGR image, and parameters are the same for every image.

Baseline. The degraded image, unprocessed.

Linear contrast stretching. The 1st and 99th percentiles are mapped to 0 and 255 and the rest is clipped. With min/max a few noisy pixels would set the range and the stretch would do almost nothing. *stretch_rgb* works on each channel separately, so each gets its own gain and colours can shift. *stretch_lab* only touches L in CIE Lab, which keeps hue and saturation.

Histogram equalization (*histeq_lab*). `cv2.equalizeHist` on L, with a and b unchanged. I skipped the per-channel version because it gives strong false colours.

Fourth method: NLM + CLAHE (*nlm_clahe*). `cv2.fastNlMeansDenoisingColored` (h = hColor = 10, template 7, search 21), then CLAHE on L (clip limit 2.0, 8×8 tiles). Global equalization uses one curve for the whole image, so it can't handle a scene that is dark on one side, and it amplifies noise in flat dark areas. CLAHE works per tile and limits the added contrast, and denoising first stops it from enhancing the noise. The downside is that NLM also smooths real detail. I ran *nlm* and *clahe* alone too, to see what each step adds.

Metrics. Against the original: PSNR (`cv2.PSNR`), MAE and my own SSIM following [4] (11×11 Gaussian window, σ = 1.5, averaged over channels, 5 px border excluded). All values are in `results/metrics.csv`.

4. Results

	img1		**img2**		**img3**		**img4**		**img5**		**mean**		
method	PSNR	SSIM	PSNR	SSIM	PSNR	SSIM	PSNR	SSIM	PSNR	SSIM	PSNR	SSIM	MAE
baseline	19.30	0.389	12.78	0.286	17.66	0.398	18.11	0.355	18.46	0.421	17.26	0.370	27.88
stretch_rgb	14.63	0.133	**15.22**	0.123	16.55	0.128	16.06	0.216	15.77	0.182	15.65	0.157	35.31
stretch_lab	15.50	0.211	14.54	0.169	16.92	0.200	18.23	0.287	18.60	0.281	16.76	0.230	30.91
histeq_lab	12.30	0.170	14.41	0.113	14.31	0.157	8.73	0.145	8.88	0.111	11.72	0.139	58.76
nlm	19.72	**0.813**	12.88	**0.636**	17.93	**0.849**	18.18	**0.511**	18.73	**0.735**	17.49	**0.709**	26.72
clahe	18.21	0.205	12.35	0.139	17.17	0.204	17.64	0.271	17.21	0.243	16.52	0.212	33.23
nlm_clahe	**21.69**	0.810	13.61	0.633	**18.77**	0.848	**19.72**	0.459	**21.47**	0.677	**19.05**	0.685	**25.61**

Table 2: PSNR (dB) and SSIM per image and method on the synthetic degradations, with the mean over the five images (MAE as mean only). Best value per column in bold.

On average, only the methods with denoising beat the untouched degraded image. *nlm_clahe* has the best mean PSNR (19.05 dB vs 17.26) and MAE (25.61 vs 27.88), and together with *nlm* it nearly doubles SSIM (0.685 and 0.709 vs 0.370). The pure contrast methods are worse than doing nothing, on average. The exception is img2, which has one very dark and one very bright mode: halving the contrast hurts it a lot, so just restoring the range gives the best PSNR (*stretch_rgb*, 15.22 dB vs 12.78).

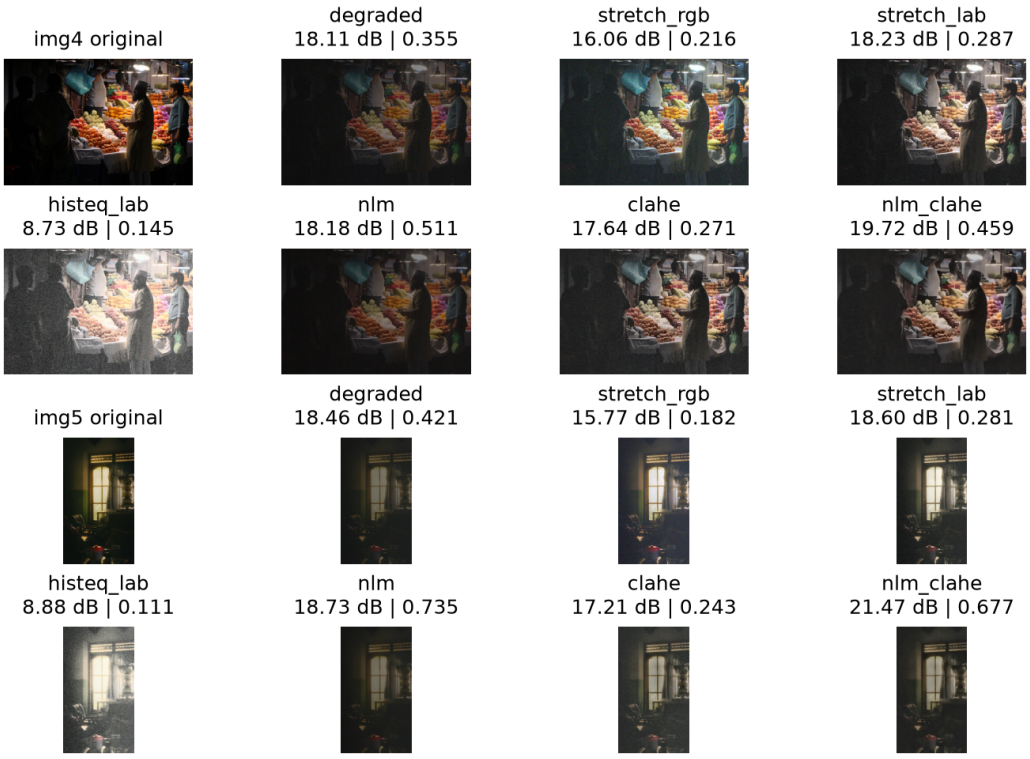


Figure 2: synthetic degradation, img4 and img5. Above each image: PSNR | SSIM with respect to the original.

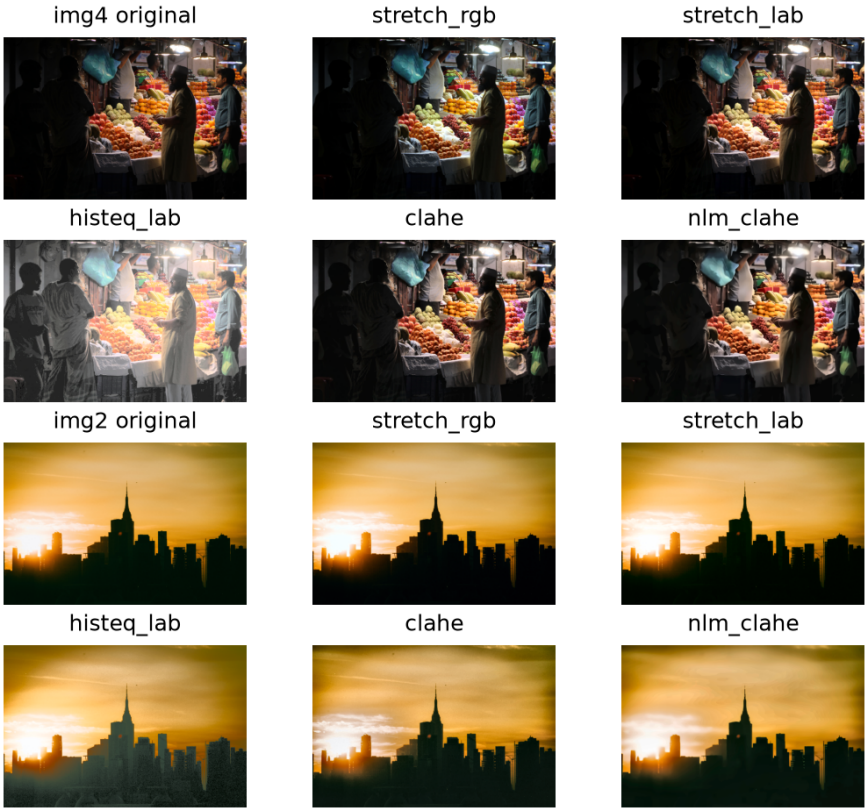


Figure 3: the same methods on the real originals of img4 and img2 (no metrics, there is no clean reference).

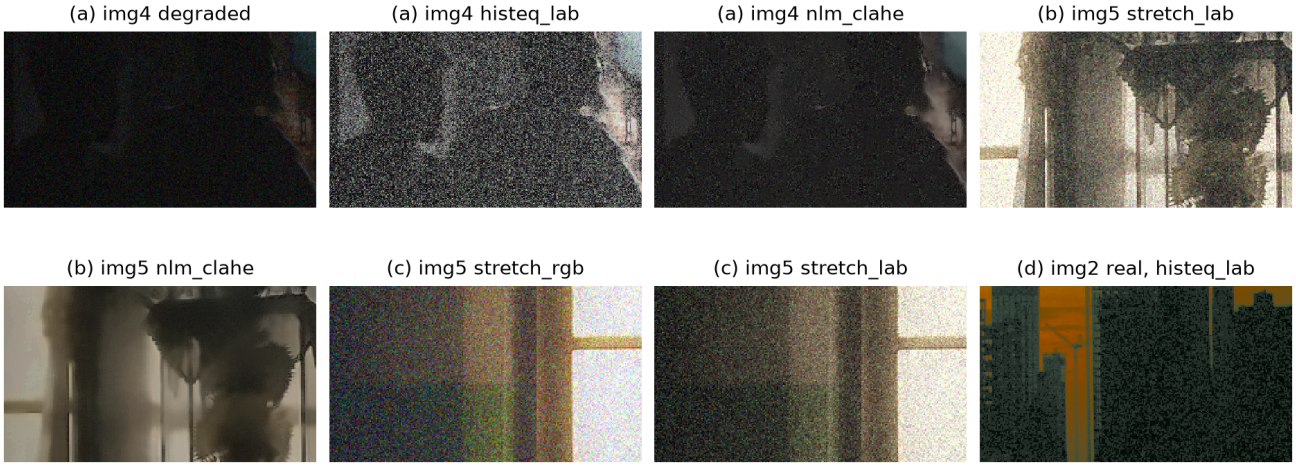


Figure 4: details. (a) shadows in img4; (b) window decoration in img5; (c) dark wall of img5 with the two stretching variants; (d) noise and compression artifacts in the dark area of img2 after global equalization.

5. Discussion

Better numbers, worse image. On img4, histogram equalization gets 8.73 dB, the worst value in the table, yet it is the degraded result where the two men on the left stand out most, despite the heavy noise (Fig. 4a). The reference is itself a dark photo, so PSNR punishes anything that brightens the shadows. img5 goes the other way: *nlm_clahe* wins clearly over *stretch_lab* (21.47 dB vs 18.60, SSIM 0.677 vs 0.281), but it wipes out the fine pattern on the window decoration, which *stretch_lab* keeps under the grain (Fig. 4b). Removing noise raises the metrics more than losing texture lowers them.

img4, night market. On the degraded version I prefer *nlm_clahe*: fruit, clothes and the people on the right are readable with little grain and the colours hold. *histeq_lab* shows more in the shadows, but it is very noisy and desaturated. On the much cleaner real photo I prefer *clahe* alone: the clothes of the men on the left appear and the fruit keeps its colour. Global HE reveals the most but washes out the area around the lamps and leaves the shadows grey and flat.

img5, dark room (synthetic). Per-channel stretching turns the dark wall blue (Fig. 4c): the percentile range is 101 levels for blue and 119 for red, so blue gets a gain of 2.5 against 2.1. I prefer *stretch_lab* here, since it keeps the hue and the window details, at the price of some grain. *histeq_lab* shows the furniture but puts a noisy grey veil on the shadows,

and *nlm_clahe* is clean but looks painted.

img2, skyline (real). Global HE brings out the buildings but turns them grey-green and amplifies noise and compression artifacts (Fig. 4d), since the dark area has few grey levels. *clahe* keeps the silhouette and adds structure to the clouds, with dark borders near the frame and around the sun. *nlm_clahe* adds banding in the sky. Stretching changes little, because the image already covers almost the full range.

Limitations. NLM with h = 10 matches the noise level, but $\alpha = 0.5$ halved the signal, so the filter is too strong for the content and oversmooths textures. A smaller h would trade that for more noise. I kept one value so the comparison stays fair. Global methods can't undo the illumination ramp, CLAHE only partly does, and percentile stretching clips lamps and the sun to white. The metrics measure fidelity to a dark reference, not visibility, and five images with one degradation level is a small test set.

Conclusion. No method wins everywhere. On noisy inputs NLM + CLAHE gave the best metrics and a reasonable look, losing some fine texture. On clean photos CLAHE alone was the most useful. I would avoid per-channel stretching because of the colour shifts, and use global HE only after denoising.

References

[1] OpenCV documentation, https://docs.opencv.org (equalizeHist, createCLAHE, fastNlMeansDenoisingColored, PSNR).

[2] A. Buades, B. Coll, J.-M. Morel, "A non-local algorithm for image denoising", CVPR 2005.

[3] K. Zuiderveld, "Contrast Limited Adaptive Histogram Equalization", Graphics Gems IV, 1994.

[4] Z. Wang, A. C. Bovik, H. R. Sheikh, E. P. Simoncelli, "Image quality assessment: from error visibility to structural similarity", IEEE Trans. Image Processing, 13(4), 2004.

[5] R. C. Gonzalez, R. E. Woods, Digital Image Processing, 4th ed., Pearson, 2018.

[6] Images, Pexels License (https://www.pexels.com/license/):
img2: https://www.pexels.com/it-it/foto/citta-alba-tramonto-punto-di-riferimento-18927434/
img3: https://www.pexels.com/it-it/foto/strada-nebbiosa-con-alberi-spogli-in-un-autunno-nebbioso-34649347/
img4: https://www.pexels.com/it-it/foto/vivace-mercato-notturno-con-prodotti-freschi-34559646/
img5: https://www.pexels.com/it-it/foto/luce-del-sole-30422461/